INDIVIDUAL PROJECT REPORT

Review 3 — Final Prototype, Report & Viva (9/10/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role in the team
Laksh Ahuja	24BDS0212	_____	Central prediction server, live dashboard, QoS scheduler, cyclone/ocean model, demo launcher and the automated test suite

Team members: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417).
Source code: https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Abstract

Cyclone warnings depend on telemetry that must cross slow, lossy marine links at exactly the moment the weather is worst. Our team built a three-tier edge–cloud system in which ocean sensor nodes and a satellite ground station send compact data to an edge gateway, which forwards it by priority to a central prediction server. In this report I describe the parts I built: the central server that repairs lost packets, fuses in-situ, satellite and forecast risk and closes the alert loop back to the gateway; the token-bucket QoS scheduler that protects critical traffic with a work-conserving bandwidth reserve; the physics-based cyclone model that drives the simulation; the live WebSocket dashboard; the one-command launcher; and the 28-test automated suite. On a congested 1200 B/s backbone the scheduler kept the 95th-percentile latency of critical packets at 356 ms compared with 49 s for first-in-first-out queuing, and at 1000 B/s it delivered 100 % of critical packets against 77.3 %. In a 75-second live run the server raised the alert at about 45 s, reconstructed 26 lost readings, concealed 6 overdue ones in real time and recovered 1 late packet.

Keywords: edge computing, quality of service, token bucket, cyclone monitoring, packet-loss recovery, FastAPI, WebSocket

1. Introduction

1.1 Problem statement

Ocean and coastal sensors that track a developing cyclone send their readings over narrow, unreliable links; when a storm approaches, the volume of important data rises just as the link degrades, so the readings that matter most are delayed or lost.

1.2 Why it matters and who is affected

India has a coastline of roughly 7,500 km and the Bay of Bengal is one of the most cyclone-prone basins in the world; the 1999 Odisha super cyclone alone caused about 10,000 deaths [2]. The WMO's 50-year disaster atlas lists tropical cyclones among the leading causes of both human and economic losses from weather extremes [1]. The people affected are coastal and island communities and fishers, who rely on timely warnings, and the disaster-management agencies that issue them. If the problem is left unsolved, a warning system can show stale or missing values during the critical hours: in our own first-in-first-out baseline on a congested link, 5 % of critical readings took longer than **49 s** to arrive, and **23 %** were lost when the link was slightly undersized (Section 5).

1.3 Objectives of my part

- Build a central server that tracks every node's sequence numbers, repairs gaps, publishes a fused cyclone-risk output and pushes the alert state back to the gateway.
- Design a backbone scheduler that serves critical traffic first without wasting capacity when the link is idle.
- Provide realistic test data (a cyclone that approaches, passes and recedes) and a dashboard that makes every mechanism visible during the demo.
- Make the whole project reproducible: one-command launch and an automated test suite.

2. Literature Review

Table 1 lists the works most relevant to my modules (networking, QoS, edge architecture and the physical storm model). The full 15-paper survey was presented in Review 1.

Table 1: Literature most relevant to my modules

Author / Year	Method used	Result	Limitation for our problem
Blake et al., 1998 [4]	DiffServ: packets marked into classes, per-hop priority	Scalable class-based QoS in IP networks	Classes are static; no link to an external risk signal
Floyd & Jacobson, 1993 [5]	RED: early random drops as the queue grows	Avoids global synchronisation, keeps queues short	Drops are class-blind; critical data can be dropped
Shreedhar & Varghese, 1996 [6]	Deficit round-robin fair queuing	O(1) fairness between flows	Fairness is the goal, not urgency; no alert mode
Shi et al., 2016 [7]	Edge computing vision and challenges	Argues for processing near data sources	Conceptual; no cyclone or lossy-link evaluation
Satyanarayanan, 2017 [8]	Cloudlets / edge tier architecture	Lower latency and bandwidth via edge tier	General purpose; no prioritisation policy
Holland, 1980 [3]	Analytic wind–pressure profile of hurricanes	Widely used parametric storm model	Needs parameters; not a telemetry system
Shelby et al., 2014; Banks & Gupta, 2014 [10, 11]	CoAP and MQTT lightweight IoT messaging	Small headers, suits constrained devices	No cyclone-aware scheduling at the gateway

Author / Year	Method used	Result	Limitation for our problem
Lantz et al., 2010 [9]	Mininet network emulation	Realistic topologies on one laptop	Tool only; QoS policy must be supplied

Gap my part fills: existing QoS schemes classify traffic statically and either drop blindly (RED) or share fairly (DRR). None ties the priority to a live cyclone-risk signal coming back from the prediction server, and none keeps a small reserve for critical packets while staying work-conserving. On the server side, the generic edge papers do not combine gap repair, real-time concealment and risk fusion with a feedback loop to the network.

3. Methodology

3.1 System overview

The system has three tiers (Figure 1): sensing nodes, an edge gateway and the central server with the dashboard. My modules are the scheduler inside the gateway, the whole central server and dashboard, and the ocean model that feeds the sensors.

FIG. 1

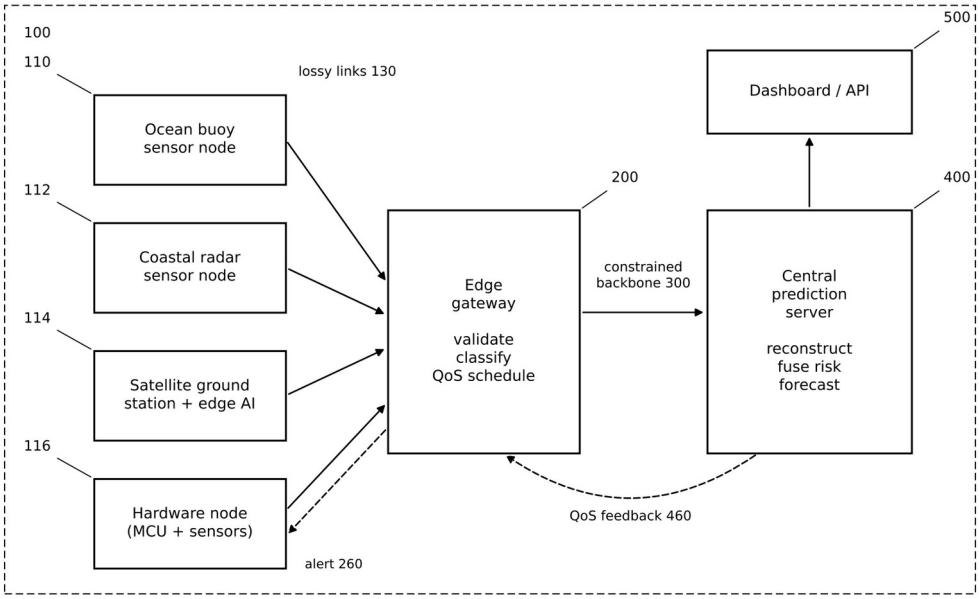


Figure 1: Overall architecture (reference numerals as in our patent specification); my modules: QoS scheduler 240 inside 200, server 400, dashboard 500

3.2 QoS scheduler design (qos.py)

The backbone is modelled as a token bucket: credit accrues at the link rate r (bytes per second) up to a cap, and each packet costs its JSON size plus 28 bytes of UDP/IP header. Three queues (CRITICAL, NORMAL, NON_ESSENTIAL) are served in strict priority. During a cyclone alert two extra rules apply (Figure 2):

- **Critical reserve:** a NORMAL or NON_ESSENTIAL packet may only be sent if at least R = 700 bytes of credit remain afterwards, so the next critical packet never waits behind it.
- **Congestion-gated shedding:** NON_ESSENTIAL packets are rejected only if more than one second of traffic is already queued; with spare capacity they are still delivered.

FIG. 7

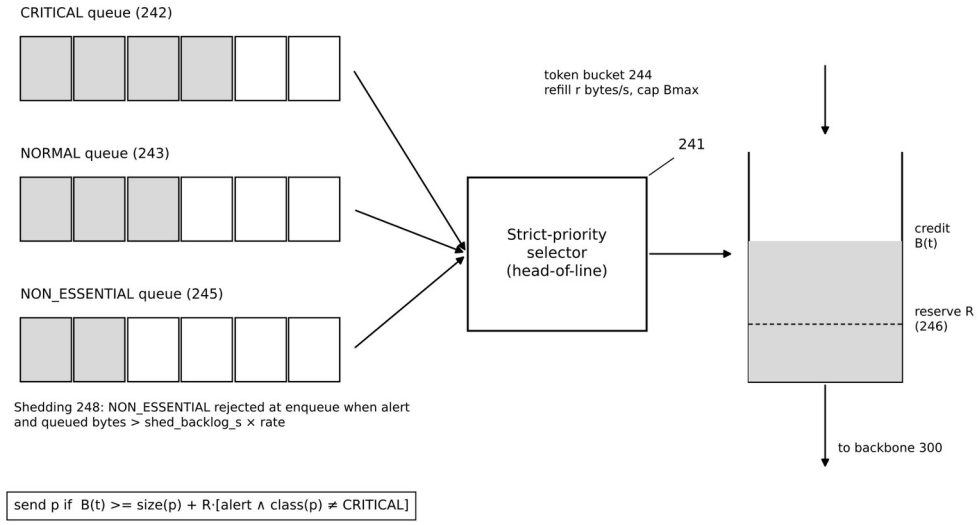


Figure 2: QoS scheduler: per-class queues, strict-priority selector, token bucket with critical reserve R

3.3 Central server design (central_server.py)

Input → process → output: a UDP listener receives each record → it is validated and routed by kind → for sensor records the server compares the sequence number with the last one (gap, duplicate, late packet or reboot) → gaps of up to 30 readings are filled by the LSTM reconstructor → a risk score is computed (50 % wind, 35 % pressure depression, 15 % pressure tendency) → the fused risk is max(in-situ, 0.9 × satellite, 0.8 × forecast) with an alert hysteresis of 60 on / 50 off → the state is served over REST/WebSocket and sent back to the gateway every 2 s. A separate 0.25 s loop publishes a provisional forecast when a node's packet is overdue by more than 2.5 times its usual interval.

FIG. 6

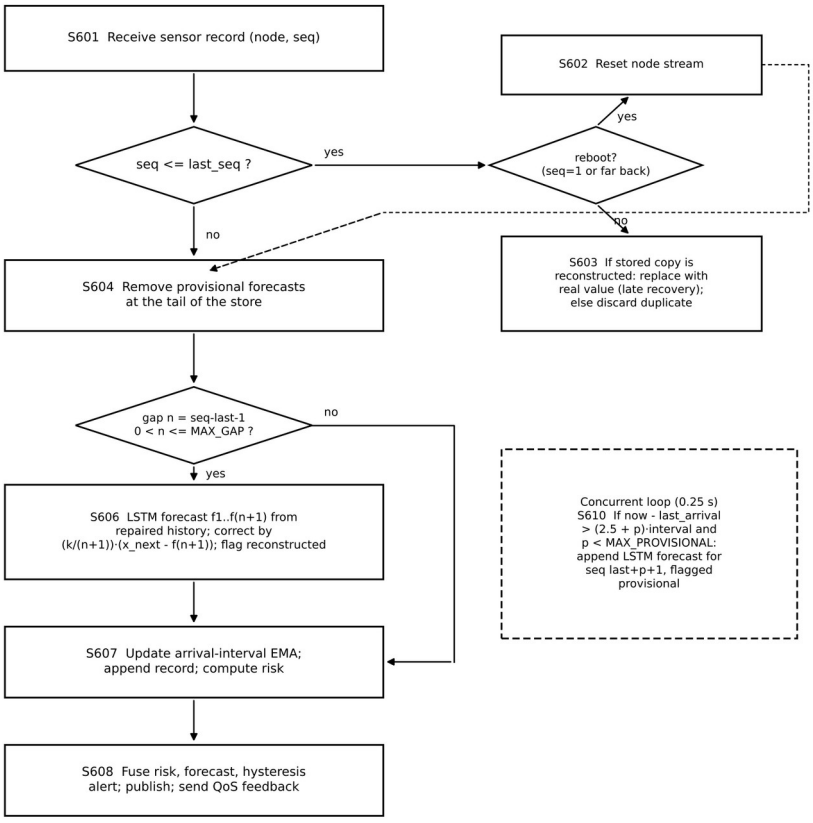


Figure 3: Server ingest, reconstruction and concealment flowchart (S601–S610)

3.4 Cyclone model (ocean_model.py)

Wind follows a modified Rankine profile and pressure follows the Holland profile [3] as a vortex approaches, passes and recedes; AR(1) gusts, a diurnal temperature cycle and rain cooling are added (Figure 4). The same model generates the LSTM training data, so the simulation and the learning use consistent physics.

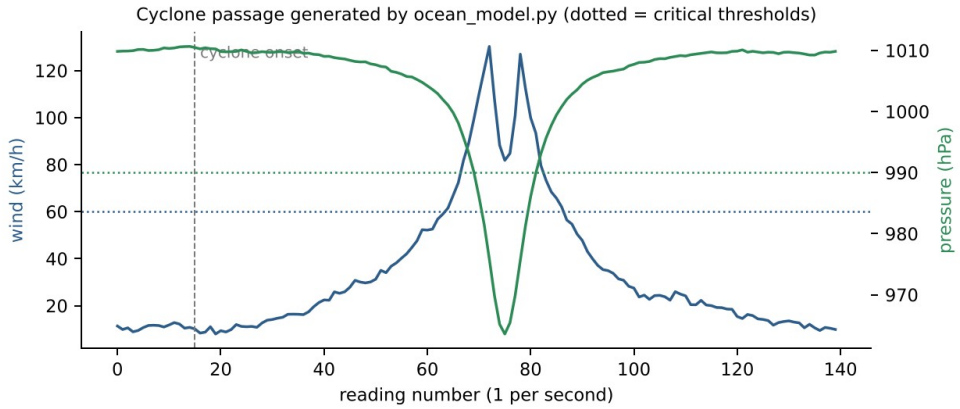


Figure 4: One cyclone passage from ocean_model.py; wind crosses 60 km/h and pressure drops below 990 hPa near closest approach

3.5 Justification of design choices

Table 2: Design choices for my modules

Decision	Alternatives considered	Chosen	Reason
Backbone scheduling	FIFO; fixed bandwidth cap per class; weighted fair queuing	Strict priority + token bucket + critical reserve	Lowest critical latency; a fixed cap measurably hurt latency below saturation; WFQ guarantees fairness, not urgency
Server framework	Flask; Node.js; Django	FastAPI + uvicorn	Async WebSocket support next to a UDP thread; automatic API docs; same language as the AI code
Transport to server	TCP; MQTT broker; UDP	UDP datagrams	Loss must be visible to be handled; no retransmission delay; MQTT kept as an embodiment
Dashboard update	Polling every 2 s; server-sent events; WebSocket	WebSocket with polling fallback	Push every 1 s; still works if the WebSocket library is missing
Test data	Random walk; recorded buoy files; physics model	Rankine + Holland physics model	Realistic storm shape with known truth; real archives planned

4. Implementation

Table 3: Files I implemented (line counts from the repository)

File	Lines	Key functions / classes	What I implemented
central_server.py	417	ServerState.ingest, _ingest_sensor, conceal_overdue, predict, udp_listener, background_loop	Gap / late / duplicate / reboot handling, real-time concealment, risk fusion, feedback, 9 REST endpoints + WebSocket
qos.py	178	TokenBucket, QoSScheduler.enqueue/_head/dequeue/next_ready_in	Strict priority, reserve, shedding, FIFO baseline, per-class statistics
ocean_model.py	146	OceanNode, CycloneParams, GilbertElliottLink	Storm physics and bursty loss channel
dashboard.html	233	render(), drawSeries(), WebSocket + polling fallback	Live banner, KPIs, tables, canvas charts, satellite thumbnail
run_demo.py	121	start(), main()	Starts 6 processes, waits for health, prints status, clean shutdown
tests/	300	28 pytest cases	Unit + integration tests for every module

The scheduler is written as pure logic with an injected clock, so the *same* class runs inside the live gateway and inside the discrete-event evaluation. The core decision is only a few lines:

```
def _head(self):
    for q in self._candidates():            # CRITICAL, NORMAL, NON_ESSENTIAL
        if q:
            _, cls, size, _ = q[0]
            reserve = self.reserve_bytes if (self.alert and cls != CRITICAL) else 0
            return q, size + reserve
    return None, None
```

The dashboard (Figure 5) receives the whole server snapshot once per second over a WebSocket; if the WebSocket library is missing it falls back to polling the `/snapshot` endpoint.



Figure 5: Live dashboard during an active alert (my implementation)

5. Testing and Results

5.1 Requirements and test setup

Table 4: Benchmarks for my modules

Requirement (benchmark)	Test setup	Result	Met?
Critical p95 latency < 1 s on a congested backbone	Discrete-event run, 300 s, 4 buoys + satellite, offered 1510 B/s, link 1000–2000 B/s	305–407 ms (QoS) vs 14–66 s (FIFO)	Yes
Critical delivery ≥ 99 % under congestion	Same, 225 critical packets per run	100 % (QoS) vs 77.3 % (FIFO at 1000 B/s)	Yes
No capacity wasted when the link is idle	Same, 2000 B/s link	Identical to FIFO, 0 shed	Yes
Alert raised during a cyclone	Live 75 s run, 3 nodes, 12 % bursty loss	Alert at ≈45 s; risk 77–100	Yes
Dashboard update ≤ 2 s	Browser on live run	WebSocket push every 1 s	Yes
All automated tests pass	pytest, 28 cases	28 passed in 1.35 s	Yes

5.2 QoS results

Table 5: Scheduler comparison (simulated clock, same code as gateway)

Backbone	Scheduler	Critical delivered	Critical avg / p95	Normal delivered
1000 B/s	FIFO	77.3 %	58584 / 65803 ms	82.4 %
1000 B/s	Priority QoS	100.0 %	90 / 407 ms	82.1 %
1200 B/s	FIFO	100.0 %	36642 / 49027 ms	92.7 %
1200 B/s	Priority QoS	100.0 %	80 / 356 ms	100.0 %
1400 B/s	FIFO	100.0 %	10210 / 14346 ms	100.0 %
1400 B/s	Priority QoS	100.0 %	79 / 305 ms	100.0 %
2000 B/s	FIFO	100.0 %	0 / 0 ms	100.0 %
2000 B/s	Priority QoS	100.0 %	0 / 0 ms	100.0 %

FIG. 13

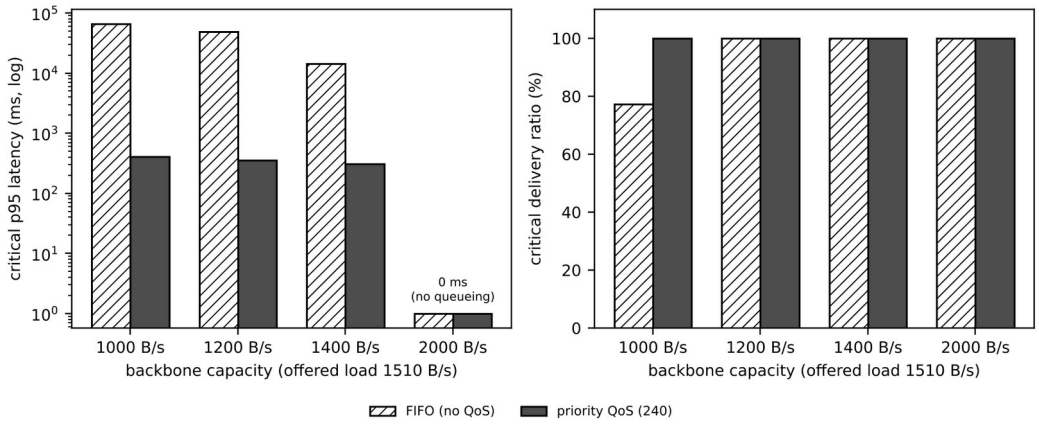


Figure 6: Critical p95 latency (log scale) and delivery ratio, FIFO vs our QoS

5.3 Live end-to-end run

Figure 7 is plotted from the status lines my launcher printed during a 75-second run (`docs/reports/evidence/live_run_output.txt`). The server received 268 packets, repaired 26 lost readings, concealed 6 overdue readings in real time, swapped in 1 late packet, and measured a gateway-to-server critical p95 latency of 388 ms; the satellite stream used 27,095 bytes instead of 52.4 MB (99.948 % less).

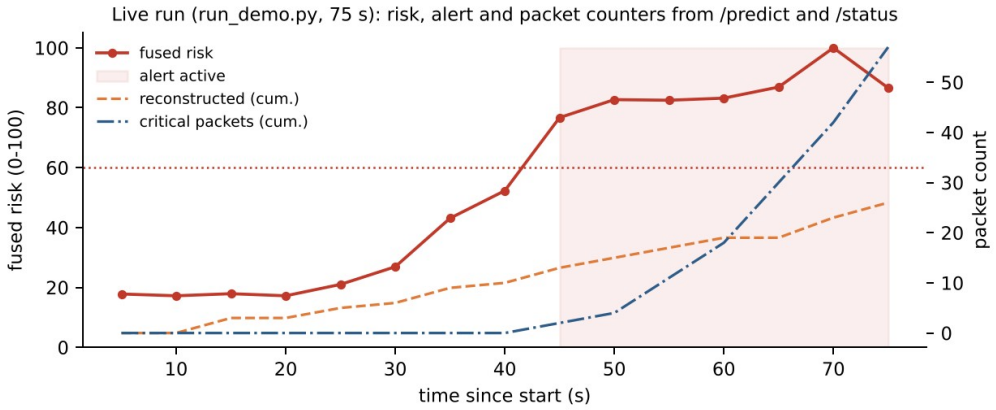


Figure 7: Live run: fused risk, alert window and cumulative counters

5.4 Failures found and fixed

Table 6: Failures and fixes in my modules

What failed	Diagnosis	Change	Retest result
Evaluation hung	`next_ready_in()` assumed	Both functions now share	Run completes; dedicated

What failed	Diagnosis	Change	Retest result
forever	a capped NORMAL queue could be skipped while `dequeue()` did not; the simulation advanced by nanoseconds	one `_head()` rule	test `test_next_ready_in_is_consistent_with_dequeue`
A 50 % cap on NORMAL traffic made critical latency *worse* below saturation	Cap built a NORMAL backlog that drained in bursts and emptied the token bucket	Replaced the cap with the 700-byte critical reserve	At 1600 B/s critical latency went from 116 ms avg back to 0
Dashboard stuck on 'polling' with empty charts	WebSocket returned 404 because the `websockets` package was missing; fallback polled `/predict`, which lacks series data	Added `/snapshot` endpoint and listed `websockets` in requirements	Live WebSocket and full fallback both verified in a browser
One of my tests failed (21 of 30 packets sent)	The test switched the alert on before queueing, so shedding correctly removed 9 packets	Test now raises the alert after the backlog exists	28/28 pass
Traceback noise on Ctrl+C in logs	Child processes received SIGINT without handlers	Clean KeyboardInterrupt handling in every component	Logs end with a clean 'stopped' line

6. Live Demonstration

1. Run `python run_demo.py` and open http://127.0.0.1:8000 (input: 3 simulated nodes + satellite).
2. 0–15 s: green banner, NORMAL traffic, flat charts.
3. ≈30–45 s: pressure falls, packets turn CRITICAL, banner turns red; the QoS table shows CRITICAL waiting far less than NORMAL / NON_ESSENTIAL.
4. Throughout: yellow dots appear where packets were lost and repaired; hollow dots for real-time guesses.
5. Close-up: `/predict` and `/metrics` in the browser; then `python -m pytest -q` (28 passed).
6. Baseline contrast: `python run_demo.py --scheduler fifo --backbone-bps 1000`.

6.1 Review 3 Video Plan (6 minutes)

Time	Content
0:00–2:00	Live demo: `run_demo.py`, dashboard from calm to alert (continuous, no cuts)
2:00–4:00	Test results: QoS table and latency chart, live-run counters, failures fixed (busy loop, 50 % cap)
4:00–5:00	Comparison with DiffServ / RED / DRR / edge papers (Section 8)
5:00–6:00	My contribution (server, scheduler, dashboard, tests) and scalability plan

The video starts with my name, register number, team number and project title, as required.

7. Feasibility (Five Dimensions)

Table 7: Feasibility of the system with emphasis on my tier

Dimension	Evidence
Technical	All benchmarks in the Section 5 benchmark table met; 28/28 tests pass; full run on one laptop core.
Economic	Software cost ₹0 (Python, FastAPI, numpy are open source). Server runs on an existing laptop; a small cloud VM would cost roughly ₹400–800 per month (our Review 1 estimate of US$5–10). Hardware nodes ≈₹800 each (see teammate's report). Bandwidth saving of 99.95 % on imagery directly reduces paid satellite airtime.
Social	Benefits coastal communities with the weakest connectivity. Formal user feedback has not yet been

Dimension	Evidence
	collected; we will record faculty/peer feedback at the review: _____
Environmental	No new hardware for the server tier; less data transmitted means less radio energy at the edge; dev boards are reusable.
Operational / Safety	One-command start, health endpoint, automatic reboot detection, input validation rejects corrupt packets; no personal data handled.

8. Comparison with Literature

Table 8: Comparison with five papers from the Review 1 survey

Parameter	Existing work	Our project	Improved?
Priority basis	Static class marking (DiffServ) [4]	Class + live alert from server feedback	Yes
Behaviour under congestion	Random early drop, class-blind (RED) [5]	Only NON_ESSENTIAL shed, only when backlog > 1 s	Yes, for critical data
Use of spare capacity	Fair share between flows (DRR) [6]	Work-conserving; reserve only during alert	Comparable
Edge architecture	Vision-level (Shi et al.) [7]	Running 3-tier prototype with measured latency	Yes (concrete)
Storm model fidelity	Holland profile with real best-track parameters [3]	Same profile with randomised parameters, not fitted to real storms	No

Where we did not improve: our storm model and all results use simulated data, and the scheduler has been tested on one gateway only; RED and DRR are proven at Internet scale, ours is not.

9. Scalability and Next Steps

- **Scale:** the server state is per node, so hundreds of nodes fit in memory; beyond that, the UDP listener can be sharded by node ID and the time-series store moved to a database (e.g. TimescaleDB).
- **Deployment:** containerise server and gateway, run the server on a cloud VM, gateways on Raspberry Pis at coastal sites; authentication on the API and HMAC-signed packets.
- **Improvements:** (1) replay real buoy archives (INCOIS / NOAA) through the pipeline; (2) adaptive reserve sized from measured critical arrival rate; (3) push alerts to SMS/mobile.

10. Progress since Review 2

Table 9: What changed in my modules between reviews

Area	At Review 2 (14/9)	Now (Review 3)
Central server	UDP listener, linear interpolation, /predict, polling HTML table	LSTM gap repair, real-time concealment, late/duplicate/reboot handling, risk fusion + forecast, feedback loop, 9 endpoints + WebSocket
QoS	Priority queue + fixed 0.5 s sleep for NORMAL	Token-bucket scheduler, 3 classes, critical reserve, congestion-gated shedding, FIFO baseline
Data / demo	Random-walk sensor values; 3 terminals started by hand	Physics-based cyclone model; one-command launcher; 28 automated tests

Overall completion: software 100 % of planned scope (all modules running and tested); validation on real buoy data not yet started.

11. My Contribution

Table 10: Division of work

Tasks I did	Tasks done by teammates
Central prediction server (central_server.py): ingest, gap/late/reboot handling, concealment, fusion, feedback, API	Anvay Sharma: Edge AI; Edge gateway; LSTM from scratch
Live dashboard (dashboard.html) with WebSocket and polling fallback	Sreehari: Hardware: ESP32 buoy firmware, LoRa buoy/receiver sketches, serial bridge, hardware guide and BOM; Lost-telemetry reconstruction; Satellite ground station
QoS scheduler (qos.py): token bucket, strict priority, critical reserve, shedding	
Cyclone/ocean physics and loss channel (ocean_model.py)	
One-command launcher (run_demo.py) and the 28-test suite (tests/)	

Evidence of my work:

- Code: the files listed in my Implementation section (https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).
- Automated tests covering my modules, all passing (Table below; full output in `docs/reports/evidence/pytest_output.txt`).
- Run logs from the live demonstration: logs/server.log (reconstruction lines), live_run_output.txt (status every 5 s, final /metrics) (excerpts in Appendix B).
- Figures in this report generated from those logs by `docs/reports/make_figures.py`.

Table 11: Tests covering my modules

Automated test (tests/test_pipeline.py)	What it checks	Result
test_cyclone_raises_wind_and_drops_pressure	Storm model: wind exceeds 60 km/h and pressure falls below 990 hPa only during the storm	PASSED
test_priority_beats_fifo_for_critical_under_congestion	Critical worst-case wait < 0.5 s with QoS, > 0.5 s with FIFO, at 2x overload	PASSED
test_alert_sheds_non_essential_only_when_congested	Housekeeping is kept on an idle link and shed only with a backlog	PASSED
test_next_ready_in_is_consistent_with_dequeue	Scheduler never reports 'ready' when dequeue would refuse (no busy loop)	PASSED
test_server_reconstructs_gap	A 3-packet gap is filled with 3 flagged estimates in sequence order	PASSED
test_server_late_packet_replaces_estimate	A late packet replaces its estimate; a repeat is a duplicate	PASSED
test_server_realtime_concealment_then_repair	Overdue node gets a provisional forecast, later replaced by gap repair	PASSED
test_server_node_reboot_starts_fresh_stream	Sequence restarting at 1 resets the node instead of being discarded	PASSED
test_predict_fuses_satellite_and_raises_alert	Satellite risk 90 raises the fused alert; bandwidth metric = 99.95 %	PASSED
test_api_endpoints	/health, /predict, /telemetry, /metrics, /snapshot, / and the WebSocket respond correctly	PASSED

> *[Insert here: screenshot of my own commits / work session and a photo of me testing — required by the rubric. Shared files (common.py, mininet_topology.py) were developed jointly.]*

12. Viva Preparation

Q: Why UDP and not TCP? Retransmission would add delay and hide loss; we expose loss and handle it with priority and reconstruction.

Q: What happens if R (the reserve) is set to 0? Critical packets can wait behind a normal packet that just used the credit; with R too large normal traffic is throttled even when idle.

Q: Why hysteresis on the alert? Without it the alert would flicker when risk hovers around 60.

Q: How does the server detect loss? From per-node sequence numbers; a jump means missing packets.

Q: What did you personally build? The server, scheduler, ocean model, dashboard, launcher and tests.

13. Conclusion

My modules turn the edge–cloud idea into a measurable system: the scheduler protects critical cyclone telemetry under congestion without wasting idle capacity, the server keeps a continuous picture despite packet loss and closes the alert loop, and the dashboard and tests make every claim demonstrable. The main limitation is that all evidence comes from simulation; validating with real buoy data is the next step.

References

[1] World Meteorological Organization, *WMO Atlas of Mortality and Economic Losses from Weather, Climate and Water Extremes (1970–2019)*, WMO-No. 1267, Geneva, 2021.

[2] India Meteorological Department, "Cyclone eAtlas" and Regional Specialised Meteorological Centre (Tropical Cyclones) reports, New Delhi. [Online]. Available: https://rsmcnewdelhi.imd.gov.in

[3] G. J. Holland, "An analytic model of the wind and pressure profiles in hurricanes," *Monthly Weather Review*, vol. 108, no. 8, pp. 1212–1218, 1980.

[4] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, "An architecture for differentiated services," IETF RFC 2475, 1998.

[5] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Trans. Networking*, vol. 1, no. 4, pp. 397–413, 1993.

[6] M. Shreedhar and G. Varghese, "Efficient fair queuing using deficit round-robin," *IEEE/ACM Trans. Networking*, vol. 4, no. 3, pp. 375–385, 1996.

[7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[8] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[9] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010.

[10] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," IETF RFC 7252, 2014.

[11] A. Banks and R. Gupta (Eds.), "MQTT Version 3.1.1," OASIS Standard, 2014.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

Appendix A — How to Run My Modules

Command	Purpose
`python run_demo.py`	Starts everything; dashboard at http://127.0.0.1:8000
`python run_demo.py --duration 90`	Headless run, prints final /metrics and gateway stats
`python run_demo.py --scheduler fifo --backbone-bps 1000`	Baseline without QoS on a tight link
`uvicorn central_server:app --port 8000`	Server only (RECON_METHOD=linear for baseline)
`python -m pytest -q`	Runs the 28 automated tests

Appendix B — Evidence from the Live Run (12/10/2026)

Unedited excerpts from the log files produced by `python run_demo.py --duration 75`:

```
# live_run_output.txt (status printed by run_demo.py every 5 s)
[    5s] risk= 17.8 alert=False rx=20 reconstructed=0 concealed=0 critical=0
[   10s] risk= 17.2 alert=False rx=38 reconstructed=0 concealed=0 critical=0
[   15s] risk= 17.9 alert=False rx=56 reconstructed=3 concealed=0 critical=0
[   20s] risk= 17.2 alert=False rx=75 reconstructed=3 concealed=0 critical=0
[   25s] risk= 21.0 alert=False rx=94 reconstructed=5 concealed=0 critical=0
[   30s] risk= 26.9 alert=False rx=111 reconstructed=6 concealed=0 critical=0
[   35s] risk= 43.1 alert=False rx=128 reconstructed=9 concealed=1 critical=0
[   40s] risk= 52.3 alert=False rx=147 reconstructed=10 concealed=1 critical=0
[   45s] risk= 76.7 alert=True  rx=163 reconstructed=13 concealed=1 critical=2
[   50s] risk= 82.7 alert=True  rx=179 reconstructed=15 concealed=2 critical=4
[   55s] risk= 82.5 alert=True  rx=197 reconstructed=17 concealed=2 critical=11
[   60s] risk= 83.2 alert=True  rx=216 reconstructed=19 concealed=2 critical=18
[   65s] risk= 86.9 alert=True  rx=232 reconstructed=19 concealed=2 critical=30
[   70s] risk=100.0 alert=True  rx=249 reconstructed=23 concealed=4 critical=42
[   75s] risk= 86.6 alert=True  rx=268 reconstructed=26 concealed=6 critical=57
# logs/server.log
[server] listening for gateway traffic on 0.0.0.0:9002 (reconstruction=lstm)
[server] BUOY_12: reconstructed 1 (seq 11)
[server] RADAR_COAST_A: reconstructed 1 (seq 11)
[server] BUOY_07: reconstructed 1 (seq 12)
[server] BUOY_12: reconstructed 1 (seq 21)
[server] BUOY_07: reconstructed 1 (seq 22)
[server] BUOY_12: reconstructed 1 (seq 27)
[server] BUOY_07: reconstructed 1 (seq 31)
```